

歩行制御の ROS 2 実装設計

歩行パラメータ生成からサーボ送信までのシーケンスと理論

ROBO-ONE Auto 2026 機 向け技術ノート (2)

2026 年 8 月 27 日

先に結論

- 歩行に関わる計算は全部 `motion` ノードの 1 プロセスに閉じ、その中の 200 Hz スレッドが「推定 → 歩行エンジン → 姿勢補償 → 逆運動学 → リンク変換 → サーボ送信」を関数呼び出しで直結する。トピックは境界にしか置かない。入るのは `/cmd_walk`・`/imu/data`・`/estop`、出るのは `/joint_states`・`/motion/state` と調整用の `/walk/debug` だけである。
- 歩行の計算本体は ROS に依存しないライブラリ `walk_core` に置き、`update(inputs, dt) → outputs` の 1 関数で呼ぶ。ノードはその入出力をトピックとサーボに配線するだけにする。
- パラメータは三層に分ける。起動時に読む静的設定 (`gait.yaml`)、歩の境界で決める 1 歩のパラメータ (着地点・ヨー・周期)、5 ms 毎に出す軌道値。指令 (v_x, v_y, ω_z) から歩のパラメータへ落とす式は §3、歩の境界と 1 周期のシーケンスは §4~§5 にある。
- 歩き始め・歩き終わりは特別扱いが要る。発散成分 ξ は静止時に両足の midpoint にあり、そこから片足の上へは「反対側の足で ZMP を押し出す」ことでしか移せない (§4.1)。止まるときは最後の 1 歩で ξ を両足の midpoint に落とす (§4.3)。

本書は前の理論ノート『6 軸脚 ×2 と IMU で二足歩行は成立するか』(以下、理論ノート) の式を実装の順序に並べ直したものである。式の導出は理論ノートに譲り、ここでは「どの量を、いつ、どこで計算し、どこへ渡すか」を書く。ノード構成の全体像は『ROS2 ノード構成の推奨アーキテクチャ』(`ros-architecture.md`) に従う。

目次

1	配置：何をどこに置くか	3
1.1	プロセス境界とトピック	3
1.2	プロセスの中の構造	3
1.3	walk_core の入出力	3
1.4	言語	4
2	パラメータの三層	4
3	指令から 1 歩のパラメータへ	5
3.1	指令の整形（毎周期）	5
3.2	支持足座標系	6
3.3	着地点とヨー	6
3.4	DCM オフセットと着地点の確定	6
3.5	歩の始点の ξ と重心軌道	7
3.6	遊脚の軌道	8
3.7	歩行パラメータ自体の決め方	8
4	歩行エンジンの状態機械	9
4.1	START：歩き始めの押し出し	9
4.2	STEP：歩の境界の処理	10
4.3	STOP：足を揃えて止まる	10
4.4	FALL・ESTOP	10
5	1 周期（5 ms）のシーケンス	10
6	状態推定の実装	12
6.1	IMU メッセージから胴体の姿勢へ	12
6.2	重心と ξ	13
6.3	接地判定と位相リセット	13
7	出力段	14
7.1	逆運動学への受け渡し	14
7.2	サーボ角と較正	14
7.3	sync write と sync read	14
7.4	publish	14
8	リアルタイム性の確保	15
9	調整とデバッグの導線	15
10	安全	16
11	実装の順番	16
12	未確定・要実測	17

1 配置：何をどこに置くか

1.1 プロセス境界とトピック

トピック	型	向き	QoS・周期
/cmd_walk	geometry_msgs/Twist	behavior, teleop → motion	Reliable, depth 1, 20 Hz
/cmd_motion	std_msgs/String	behavior, teleop → motion	Reliable, イベント
/estop	std_msgs/Bool	teleop → motion	Reliable, イベント
/imu/data	sensor_msgs/Imu	imu_filter → motion	SensorData, 200 Hz
/joint_states	sensor_msgs/JointState	motion → ui, 記録	Reliable, 10 Hz
/motion/state	std_msgs/String	motion → behavior, ui	Reliable, 状態変化時
/walk/debug	std_msgs/Float64MultiArray	motion → PlotJuggler, 記録	BestEffort, 50~200 Hz

表 1: motion ノードの境界。/walk/debug は調整用で、本番では止めてよい。

/cmd_walk は linear.x, linear.y, angular.z だけを使う。機体座標系、SI 単位である。サーボバス (/dev/ttyACM0, ttyACM1) はトピックを通らず、motion ノードがプロセス内で直接叩く。

1.2 プロセスの中の構造

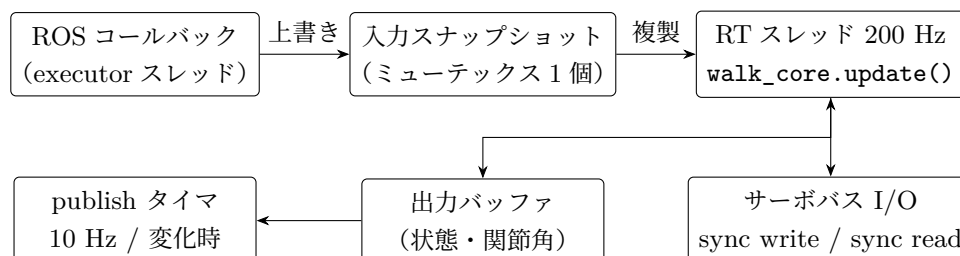


図 1: motion ノードの内部。ROS のコールバックはメッセージを最新値として置くだけで計算しない。RT スレッドは毎周期スナップショットを 1 回複製して以降ロックを取らない。

RT スレッドを ROS のタイマにしない理由は、executor の起床時刻が DDS の受信処理に引きずられて数 ms ぶれるからである。自前のスレッドで絶対時刻のデッドラインまで眠る (§8)。

1.3 walk_core の入出力

walk_core は乱数も時計も持たず、渡された *dt* と入力だけで決定的に動く。これで同じ入力列を流せば同じ出力が出るので、ROS なしの単体テストと bag 再生による再現ができる。

量	出どころ
入力	胴体姿勢 R_{WB} 、角速度 ω_B 、比力 f_B 、時刻 関節角実測 q_m 、負荷 指令 (v_x, v_y, ω_z) 、受信時刻 非常停止、技指令
出力	関節角目標 q_{cmd} (脚 12 + 腕 6) トルクリミット比率 (軸ごと) 状態 (IDLE, START, STEP, STOP, FALL, ESTOP)、位相、支持脚 デバッグ配列 $(\xi, p, \phi, \theta, \text{ゲイン適用量, 周期遅れ})$
	$/imu/data$ を胴体座標系に直したものの (§6.1) sync read (読めない周期は直前値) $/cmd_walk$ $/estop, /cmd_motion$ 逆運動学の後、リンク変換の前 着地剛性スケジュール $/motion/state$ と行動層の同期用 $/walk/debug$

1.4 言語

最初は Python で書く。サーボドライバ `feetech_bus.py` が既に Python であること、1 周期の計算量が小さいこと (閉形式 IK $\times$ 2、DCM の指数関数、 4×4 行列が十数個)、ゲイン調整の回復が速いことが理由である。ただし `walk_core` の入出力を上表の形に固定し、内部で行列ライブラリに依存しない書き方をしておけば、C++ への移植は機械的に済む。移植の判断は §8 の周期ジッタ計測で決める。

2 パラメータの三層

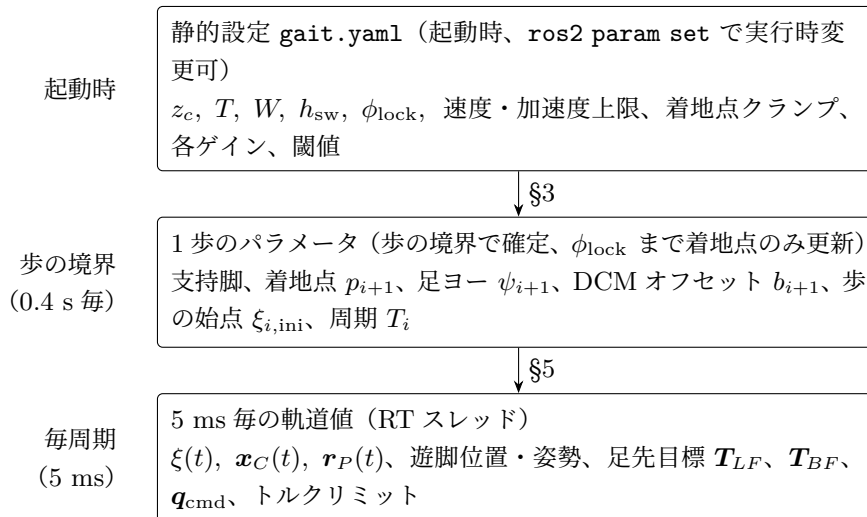


図 2: パラメータの三層。上の層ほど変更頻度が低い。指令 (v_x, v_y, ω_z) は中段に入る。

名前	初期値	単位	効く場所
<code>z_c</code>	0.16	m	$\omega = \sqrt{g/z_c}$ 、骨盤高さ。発熱試験で下限を決める
<code>T_step</code>	0.40	s	$e^{\omega T}$ 。短いほど押しに強い
<code>W</code>	0.08	m	左右の足間隔。CAD の股間隔から決める
<code>swing_height</code>	0.02	m	遊脚の頂点高さ
<code>swing_lock_phase</code>	0.70	–	着地点の凍結位相
<code>td_overdrive</code>	0.004	m	名目床面より下へ突き抜ける量
<code>td_speed_max</code>	0.10	m/s	着地直前の降下速度上限
<code>v_max</code>	(0.15, 0.08, 0.8)	m/s, m/s, rad/s	指令の飽和
<code>a_max</code>	(0.3, 0.2, 2.0)	m/s ² , m/s ² , rad/s ²	指令のレート制限
<code>step_clamp</code>	前後 0.04、外 0.045、内 0.015	m	名目着地点からの許容ずれ
<code>start_pushoff_max</code>	0.15	s	押し出しの最長時間
<code>k_d_ankle</code>	(0.05, 0.05)	rad/(rad/s)	足首ピッチ・ロールのジャイロ減衰
<code>k_p_ankle</code>	(0, 0)	rad/rad	足首の角度比例（後から入れる）
<code>k_torso</code>	(0, 0)	–	体幹水平化のゲイン（ロール・ピッチ）
<code>ankle_clamp</code>	0.12	rad	足首オフセットの飽和
<code>gate_pre_td</code>	0.03	s	着地前のゲイン半減窓
<code>td_load_thresh</code> , <code>td_jerk_thresh</code>	要校正	–	接地判定
<code>k_dcm</code>	1.0	–	踏み出し補正のゲイン
<code>torque_soft</code>	0.6	–	着地脚の膝・足首のトルクリミット比
<code>fall_tilt</code>	0.7	rad	転倒判定
<code>cmd_timeout</code>	0.5	s	指令途絶で停止に入る
<code>loop_hz</code>	200	Hz	RT スレッド周期

表 2: `gait.yaml` の主要項目。ゲインは全部 0 から始められる値にしてあり、0 なら純粋なフィードフォワードになる。

3 指令から 1 歩のパラメータへ

3.1 指令の整形（毎周期）

受信した指令 $\mathbf{v}_{\text{cmd}} = (v_x, v_y, \omega_z)$ を、そのまま使わず二段で整える。まず飽和とレート制限で

$$\mathbf{v}_k = \mathbf{v}_{k-1} + \text{clamp}(\text{clamp}(\mathbf{v}_{\text{cmd}}, -\mathbf{v}_{\text{max}}, \mathbf{v}_{\text{max}}) - \mathbf{v}_{k-1}, -\mathbf{a}_{\text{max}}\Delta t, \mathbf{a}_{\text{max}}\Delta t) \quad (1)$$

とする。歩幅が 1 歩ごとに跳ばないようにするためである。次に前進と旋回の同時要求を楕円で制限する。

$$s = \sqrt{\left(\frac{v_x}{v_{x,\text{max}}}\right)^2 + \left(\frac{\omega_z}{\omega_{z,\text{max}}}\right)^2}, \quad s > 1 \Rightarrow (v_x, \omega_z) \leftarrow (v_x, \omega_z)/s \quad (2)$$

大きな旋回と大きな前進を同時に出すと遊脚の到達域を超えるため、旋回を優先したいなら v_x だけを縮める。指令が `cmd_timeout` 以上届かなければ $\mathbf{v}_{\text{cmd}} = \mathbf{0}$ とみなし、§4.3 の停止に入る。

3.2 支持足座標系

歩 i の間の計算は全て支持足座標系 $\{S_i\}$ で行う。原点は支持足の足裏中心、 x は足のヨー方向、 z は鉛直上向きである。歩の境界で新しい支持足に乗り換えるとき、状態量 (ξ , $\mathbf{x}_C$ 、遊脚位置) を

$$\mathbf{r}^{S_{i+1}} = \mathbf{R}_z(-\Delta\psi_i) (\mathbf{r}^{S_i} - \mathbf{p}_{i+1}^{S_i}) \quad (3)$$

で移す。 $\mathbf{p}_{i+1}^{S_i}$ は着地した足の位置、 $\Delta\psi_i$ はそのヨー角である。この乗り換えの累積が脚オドメトリになり、行動層に渡すヨー角はこれではなくジャイロ積分を使う。

3.3 着地点とヨー

1 歩の移動量を、整形後の指令と周期から

$$L_x = v_x T, \quad L_y = v_y T, \quad \Delta\psi = \omega_z T \quad (4)$$

と置く。歩 i の支持足が右足なら次は左足で、その名目着地点は支持足座標系で

$$\mathbf{p}_{i+1}^{\text{nom}} = \mathbf{R}_z\left(\frac{\Delta\psi}{2}\right) \begin{pmatrix} L_x \\ s_{i+1}W + L_y \end{pmatrix}, \quad \psi_{i+1} = \Delta\psi, \quad s_{i+1} = \begin{cases} +1 & \text{次が左足} \\ -1 & \text{次が右足} \end{cases} \quad (5)$$

である。 $\mathbf{R}_z(\Delta\psi/2)$ を掛けるのは、骨盤の向きが両足のヨーの平均になるので、移動量を骨盤の向きで測るためである。

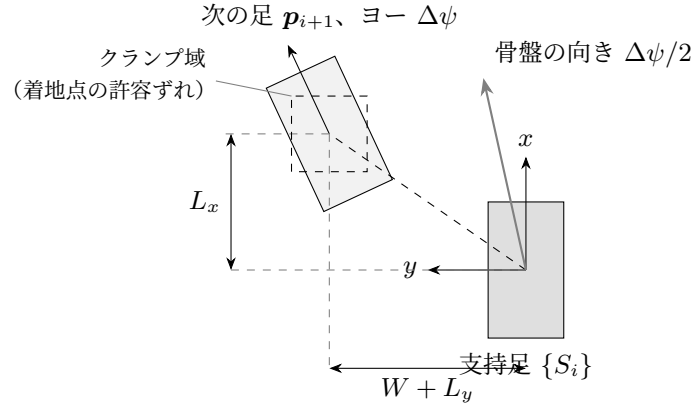


図 3: 支持足座標系で見た次の着地点。移動量は骨盤の向き（両足ヨーの平均）で測り、着地点はクランプ域の中に収める。図は $\Delta\psi$ を誇張している。

3.4 DCM オフセットと着地点の確定

理論ノート §3.4 の関係を、前後と左右で分けて書く。前後は毎歩同じ L_x なので

$$b_x = \frac{L_x}{e^{\omega T} - 1} \quad (6)$$

である。左右は L_y の符号が毎歩反転する周期 2 歩の軌道になる。 $b_1 e^{\omega T} = L_1 + b_2$ 、 $b_2 e^{\omega T} = L_2 + b_1$ 、 $L_2 = -L_1$ を解くと

$$b_1 = \frac{L_1}{e^{\omega T} + 1}, \quad b_2 = \frac{L_2}{e^{\omega T} + 1} \quad (7)$$

であり、左右成分の分母は +1 になる。まとめて、次の歩のオフセットは

$$\mathbf{b}_{i+1} = \begin{pmatrix} \frac{L_x}{e^{\omega T} - 1} \\ \frac{s_{i+2}W + L_y}{e^{\omega T} + 1} \end{pmatrix} \quad (8)$$

である。 $s_{i+2}W + L_y$ は着地点 $\mathbf{p}_{i+1}$ から見た「その次の足」までの左右距離である。 $W = 0.08$ m、 $e^{\omega T} = 22.9$ なら左右のオフセットは 3.3 mm、前後は $L_x = 0.06$ m で 2.7 mm になる。

歩 i の終端の ξ を

$$\hat{\xi}_{i,\text{eos}} = \mathbf{p}_i + \left(\hat{\xi}(t) - \mathbf{p}_i \right) e^{\omega(T-t)} \quad (9)$$

で予測し、着地点を

$$\mathbf{p}_{i+1} = \mathbf{p}_{i+1}^{\text{nom}} + k_{\text{dcm}} \left(\hat{\xi}_{i,\text{eos}} - \xi_{i,\text{eos}}^{\text{nom}} \right), \quad \xi_{i,\text{eos}}^{\text{nom}} = \mathbf{p}_{i+1}^{\text{nom}} + \mathbf{b}_{i+1} \quad (10)$$

で決める。 $k_{\text{dcm}} = 1$ なら理論ノートの $\mathbf{p}_{i+1} = \hat{\xi}_{i,\text{eos}} - \mathbf{b}_{i+1}$ そのものである。 $\hat{\xi}(t)$ に計画値を入れれば純粋なフィードフォワード、推定値を入れれば踏み出し補正になり、式は変わらない。この切替が k_{dcm} と「 ξ の出どころ」の 2 つのスイッチで、層を 1 つずつ有効化できる理由である。

着地点は式 (10) の後に必ずクランプする。

$$\mathbf{p}_{i+1} \leftarrow \mathbf{p}_{i+1}^{\text{nom}} + \text{clamp}(\mathbf{p}_{i+1} - \mathbf{p}_{i+1}^{\text{nom}}, -\mathbf{c}, \mathbf{c}) \quad (11)$$

$\mathbf{c}$ は前後 0.04 m、左右は外側 0.045 m・内側 0.015 m である。内側が狭いのは脚同士の干渉のためである。式 (9)～クランプは毎周期計算し、位相 $\phi_{\text{lock}} = 0.7$ で凍結する。

3.5 歩の始点の ξ と重心軌道

歩の境界で $\xi_{i,\text{ini}}$ を取り、その歩の間は

$$\xi(t) = \mathbf{p}_i + (\xi_{i,\text{ini}} - \mathbf{p}_i) e^{\omega(t-t_i)}, \quad \dot{\mathbf{x}}_C = \omega(\xi - \mathbf{x}_C) \quad (12)$$

で ξ と重心を進める。重心は式 (12) を周期 Δt で積分する ($\mathbf{x}_C \leftarrow \mathbf{x}_C + \omega(\xi - \mathbf{x}_C)\Delta t$ で足りる)。 $\xi_{i,\text{ini}}$ に何を入れるかも切替式で、フィードフォワードなら前の歩の $\xi_{i-1,\text{eos}}$ の計画値を式 (3) で移したものの、踏み出し補正を有効にしているなら推定値である。推定値を入れると、歩の頭で ξ の軌道が実際の状態から出発し直すので、ずれは次の着地点で吸収される。

両脚支持を明示的に入れる場合は、歩の頭 T_{ds} の間だけ ZMP 参照を前の支持点から線形に移す。 $\mathbf{p}(t) = \mathbf{p}_{i-1} + (\mathbf{p}_i - \mathbf{p}_{i-1})t/T_{\text{ds}}$ に対して $\dot{\xi} = \omega(\xi - \mathbf{p})$ を積分すると

$$\xi(T_{\text{ds}}) = \mathbf{p}_{i-1} + (\xi_{i,\text{ini}} - \mathbf{p}_{i-1}) e^{\omega T_{\text{ds}}} - (\mathbf{p}_i - \mathbf{p}_{i-1}) \left[\frac{e^{\omega T_{\text{ds}}} - 1}{\omega T_{\text{ds}}} - 1 \right] \quad (13)$$

であり、以後は $\mathbf{p}_i$ 一定で式 (12) に戻る。式 (9) の予測もこの分だけ補正する。最初の実装では $T_{ds} = 0$ で始めてよい。ZMP の飛びは実機では数十 ms の遷移になり、その差はフィードバックが吸収する。

3.6 遊脚の軌道

歩 i の遊脚は、歩の頭の位置 $\mathbf{r}_0$ から $\mathbf{p}_{i+1}$ へ向かう。正規化時間 $\tau = (t - t_i)/T$ に対して

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad (14)$$

$$\mathbf{r}_{sw,xy}(\tau) = \mathbf{r}_{0,xy} + s(\tau) (\mathbf{p}_{i+1} - \mathbf{r}_{0,xy}), \quad (15)$$

$$z_{sw}(\tau) = \begin{cases} h_{sw} s(\tau/0.45) & \tau < 0.45 \\ h_{sw} (1 - s(\frac{\tau-0.45}{0.55})) - z_{od} s(\frac{\tau-0.45}{0.55}) & \tau \geq 0.45 \end{cases} \quad (16)$$

$$\psi_{sw}(\tau) = \psi_0 + s(\tau) (\psi_{i+1} - \psi_0) \quad (17)$$

とする。 z_{od} は突き抜け量 `td_overdrive` である。 $\mathbf{p}_{i+1}$ が更新されるたびに終点だけ差し替え、 $s(\tau)$ は進め続ける。 $\tau > 0.45$ の降下区間では鉛直速度を `td_speed_max` で飽和させ、接地を検知したら z_{sw} をその場で凍結する。遊脚の足裏姿勢は $\{L\}$ で水平、ヨーは ψ_{sw} である。

3.7 歩行パラメータ自体の決め方

静的設定の中で機体から決めるものは、次の順で確定する。

1. W : CAD の股関節の左右間隔に等しくする。それより狭いと遊脚が支持脚をまたぐ
2. z_c : 可動域で許される上限から始め、立位+スクワット連続の温度上昇率で下限を決める
3. L_x の上限 : 遊脚の到達域。着地の瞬間に骨盤はほぼ両足の中央にあるので、股関節から着地点までの距離

$$\sqrt{\left(\frac{L_x}{2}\right)^2 + \left(\frac{W - w_{hip}}{2} + |L_y|\right)^2} + z_c^2 \leq 0.98 (l_{th} + l_{sh}) \quad (18)$$

を満たす範囲。 w_{hip} は股関節の左右間隔、 l_{th}, l_{sh} は大腿・下腿長で、いずれも IK タスクで CAD から拾う

4. T : 短いほど $e^{\omega T}$ が下がって押しに強いが、遊脚の関節角速度が上がる。5 次多項式の最大速度は平均の 1.875 倍なので、股・膝の角速度はおよそ $1.875 L_x / (T l_{th})$ になり、サーボの無負荷速度 (STS3215 で 0.222 s/60°、負荷時はさらに落ちる) の半分以下に収める
5. h_{sw} : リングは平坦なので 0.02 m から。足先のたわみ分だけ上げる

上の順にしているのは、 W と z_c が機体で決まる制約で、 L_x と T がその中で選ぶ自由度だからである。

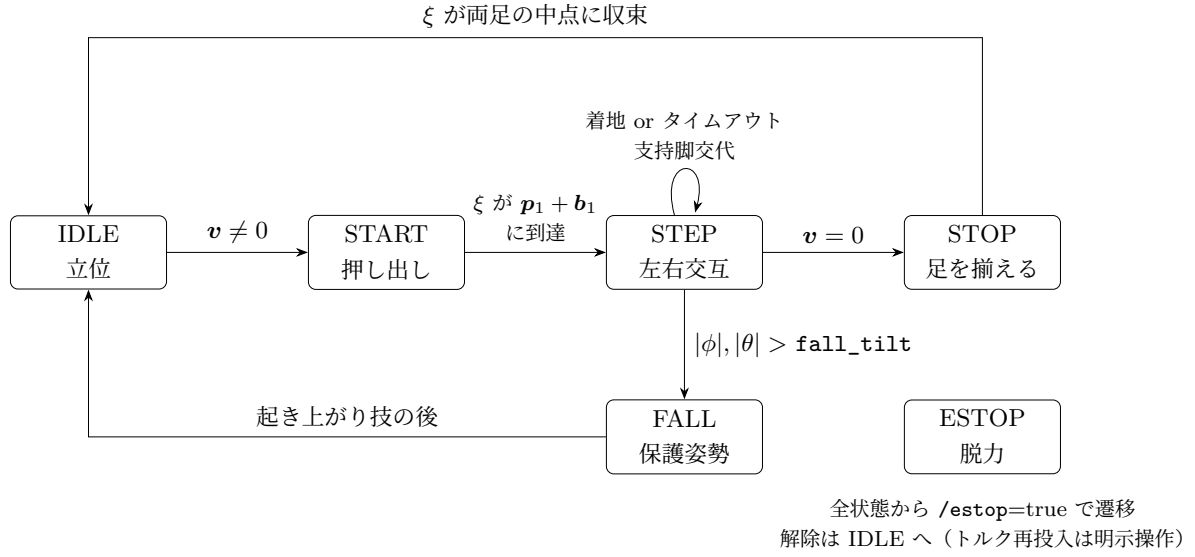


図 4: 歩行エンジンの状態。STEP の中で支持脚が交代する。FALL と ESTOP はどの状態からでも入る。

4 歩行エンジンの状態機械

4.1 START：歩き始めの押し出し

立位では ξ は両足の midpoint $\mathbf{m}$ にある。最初の支持足を左 ($y = +W/2$) とすると、歩き出すには ξ を $\mathbf{p}_L + \mathbf{b}_1$ まで運ばなければならない。ZMP を左足に置くと $\dot{\xi} = \omega(\xi - \mathbf{p}_L)$ で ξ は左足から遠ざかるので、逆効果である。ZMP は右足に置く。 $\xi(0) = \mathbf{m}$ 、 $\mathbf{p} = \mathbf{p}_R$ で

$$\xi_y(t) = -\frac{W}{2} + \frac{W}{2}e^{\omega t} \quad (19)$$

となり、 ξ は左足へ向かって走る。これが $\mathbf{p}_L + \mathbf{b}_1$ に達する時刻は、 $b_{1,y} = -W/(e^{\omega T} + 1)$ (次の足は右) を入れて

$$-\frac{W}{2} + \frac{W}{2}e^{\omega T_0} = \frac{W}{2} - \frac{W}{e^{\omega T} + 1} \Rightarrow T_0 = \frac{1}{\omega} \ln \left(2 - \frac{2}{e^{\omega T} + 1} \right) \quad (20)$$

である。 $e^{\omega T} = 22.9$ なら $T_0 = 0.083$ s になる。この間の重心は $x_C = \mathbf{p}_R + \frac{W}{2} \cosh \omega t$ で、 T_0 の終わりに左へ 9 mm 動き、0.22 m/s の横速度を持つ。つまり「押し出し」の実体は、右足で床を押して重心を左足の上へ加速する動作である。

実装では時間ではなく条件で遷移する。START では両足を床に置いたまま ZMP 参照を $\mathbf{p}_R$ に置き、式 (12) で ξ と重心を進め、 ξ_y が $\mathbf{p}_L + \mathbf{b}_1$ の y を越えた周期で STEP に入る。サーボの遅れで実機の重心が計画より遅れても、踏み出し補正が有効なら推定 ξ で判定されるので遷移が遅れるだけで済む。start_pushoff_max を超えたら異常として立位へ戻す。前後の成分 $b_{1,x}$ は押し出しでは作らず、最初の歩の着地点に式 (10) で吸収させる。

4.2 STEP：歩の境界の処理

歩の境界は、接地検知 (§6.3) か、位相 $\phi = 1.2$ のタイムアウトで起きる。境界で次の順に処理する。

1. 遊脚の鉛直目標を凍結し、支持脚を交代する。式 (3) で ξ ・重心・旧支持足の位置を新しい支持足座標系に移す
2. 整形後の指令から $L_x, L_y, \Delta\psi$ を取り、式 (5) で名目着地点 $\mathbf{p}_{i+1}^{\text{nom}}$ 、式 (8) で $\mathbf{b}_{i+1}$ を出す
3. $\xi_{i,\text{ini}}$ を確定する (計画値または推定値)
4. 遊脚の始点 $\mathbf{r}_0$ を、いま床を離れる足の現在位置にする
5. 着地剛性スケジュールを解除し、ゲート (§9) を再設定する
6. /motion/state に位相リセットと支持脚を通知する

$v = 0$ になった歩の境界では STOP へ移る。

4.3 STOP：足を揃えて止まる

止まるには、最後に ξ が両足の midpoint に落ち着き、ZMP をそこに置ければよい。両足が床にあれば ZMP は両足の間のどこにでも置ける。最後の歩 N の着地点を支持足の真横 $\mathbf{p}_N = \mathbf{p}_{N-1} + (0, s_N W)$ に固定し、歩 $N-1$ の終端の ξ が中点 $\mathbf{m}_N = (\mathbf{p}_{N-1} + \mathbf{p}_N)/2$ に来るように、歩 $N-1$ のオフセットを

$$\mathbf{b}_{N-1} = (\mathbf{m}_N - \mathbf{p}_{N-1}) e^{-\omega T} \quad (21)$$

にする。これは式 (8) の「その次の足」を中点に置き換えただけで、歩 $N-1$ の着地点は式 (10) でそのまま決まる。歩 N で足が着いたら ZMP 参照を $\mathbf{m}_N$ に置き、 $|\xi - \mathbf{m}_N|$ が閾値を下回ったら IDLE へ戻る。 ξ が中点からずれて着いた場合は、ZMP を両足の間で ξ の位置に置けば静止する。両足の外へ出ていたらもう 1 歩踏む (STEP へ戻る)。

4.4 FALL・ESTOP

$|\phi|$ か $|\theta|$ が fall_tilt を越えたら FALL に入り、歩行を止めて保護姿勢の技を再生する。姿勢が戻ったら起き上がりの技に続け、終わったら IDLE へ戻る。/estop は全状態に優先し、受信した周期のうちにトルクを切る。解除してもトルクの再投入は自動では行わず、/cmd_motion の home を明示的に受けて IDLE へ戻る。

5 1 周期 (5 ms) のシーケンス

RT スレッドの 1 周期を、呼び出す関数の順に書く。括弧内は理論ノートの対応節である。

1. 入力の複製。スナップショットから IMU・指令・非常停止を写す。前周期の sync read の結果

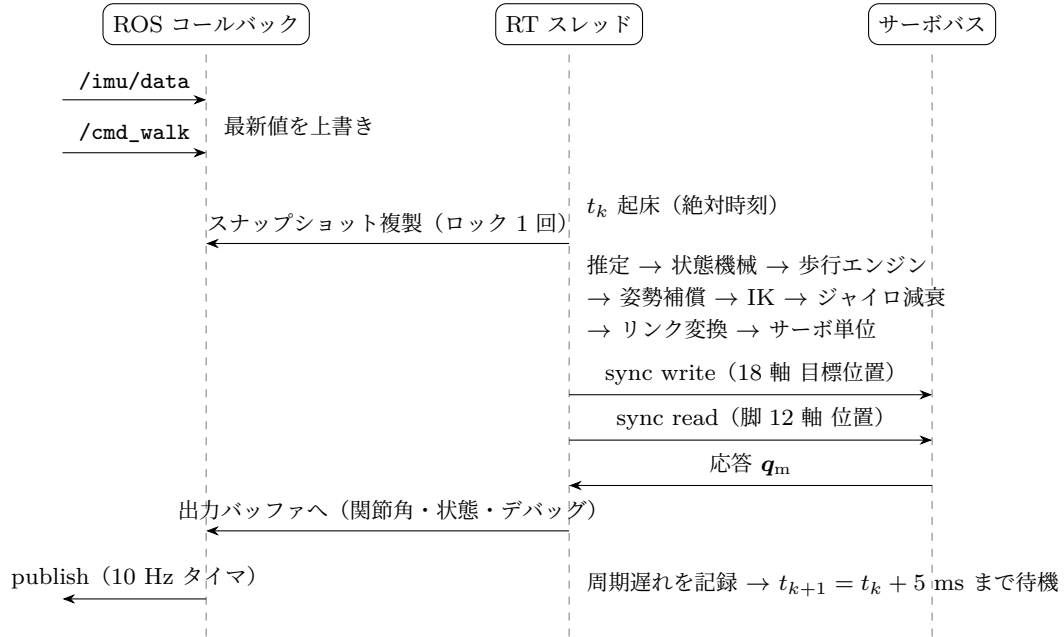


図 5: 1 周期のシーケンス。サーボの応答は次の周期の推定に使う。sync read が周期内に収まらなければ 2 周期に 1 回に落とす (§8)。

q_m と負荷も揃える

2. **姿勢の変換。** /imu/data の四元数と角速度を胴体座標系に直し、 ϕ, θ, ω_B を得る (§6.1)
3. **非常停止・転倒判定。** 該当すれば状態を切り替え、以降を飛ばしてトルクオフか保護姿勢を出す
4. **重心と ξ の推定。** 支持足基準の FK と R_{WB} から x_C 、差分と LPF から $\dot{x}_C$ 、 $\hat{\xi} = x_C + \dot{x}_C/\omega$ (理論ノート §1.2)
5. **接地判定と位相更新。** 負荷・加速度・位相のスコアで着地を判定し、歩の境界なら §4 の境界処理を行う。位相 $\phi \leftarrow \phi + \Delta t/T$
6. **着地点の更新。** $\phi < \phi_{\text{lock}}$ なら式 (9)~クランプで p_{i+1} を更新
7. **ξ ・重心・遊脚の軌道値。** 式 (12) と遊脚の式で今周期の値を出す。骨盤位置 $r_P = x_C - r_{PC}$ 、高さ z_C 、ヨーは両足ヨーの平均
8. **足先目標 T_{LF} 。** 支持足は床に固定した位置 $p_i - r_P$ 、遊脚は軌道値。どちらも水平な胴体座標系 $\{L\}$ で書く
9. **姿勢補償。** $R_{BL} = (R_x(k_r \hat{\phi}) R_y(k_\theta \hat{\theta}))^\top$ で $T_{BF} = T_{BL} T_{LF}$ (理論ノート §3.6、§4.4)
10. **逆運動学。** 両脚の T_{BF} から関節角 q 。到達域外なら着地点側を縮めて再計算し、歩行は止めない
11. **関節オフセット。** 支持脚の足首に $-k_d \omega - k_p(\phi, \theta)$ を加え、ankle_clamp で飽和。ゲート中はゲイン半減 (§9)
12. **腕と技の合成。** 腕は対側脚に同期した肩ピッチの逆振り。技再生中は脚も技の値で上書きする

13. リンク変換とサーボ単位。膝リンク・足首パラレルリンクの変換層で関節角をサーボ角へ、次に較正表でティック値へ (§7.2)
 14. トルクリミット。着地脚の膝・足首の比率を決める (着地前後だけ `torque_soft`)
 15. `sync write`、`sync read`。2 バス並列で書き、脚の位置を読む
 16. 出力バッファ。関節角・状態・デバッグ配列を書く。publish はしない
 17. 待機。周期遅れを記録し、次の絶対時刻まで眠る
- 擬似コードにすると次のようになる。

```
def rt_loop():
    t_next = monotonic()
    while running:
        t_next += DT
        sleep_until(t_next)          # 絶対時刻。遅れは蓄積しない
        late = monotonic() - t_next  # 周期ジッタの記録
        inp = snapshot.copy()        # ロックはここ 1 回
        est = estimator.update(inp, q_meas, DT)  # 姿勢, x_C, xi, 接地
        out = engine.update(est, inp.cmd, inp.estop, DT)  # 足先目標, 状態
        q = ik_both_legs(compensate(out.T_LF, est.R_WB, gains))
        q = add_ankle_damping(q, est, gains, out.gate)
        q = merge_arms_and_motions(q, out)
        servo = to_servo_ticks(link_convert(q), calib)
        bus.sync_write(servo, out.torque_ratio)
        q_meas = bus.sync_read_positions()  # 間に合わなければ隔周期
        outbox.write(q, out.state, est.debug, late)
```

6 状態推定の実装

6.1 IMU メッセージから胴体の姿勢へ

`/imu/data` の `orientation` は、Madgwick フィルタが出す IMU 座標系 $\{I\}$ の姿勢 $\mathbf{R}_{WI}$ である。ヨーは磁気を使わないので任意で、ロール・ピッチだけを信じる。IMU は RealSense の中にあり、その光学座標系は「 x 右、 y 下、 z 前」で胴体座標系と一致しない。取付の回転 $\mathbf{R}_{BI}$ を挟んで

$$\mathbf{R}_{WB} = \mathbf{R}_{WI} \mathbf{R}_{BI}^{\top} \quad (22)$$

とする。 $\mathbf{R}_{BI}$ は図面から置くより、機体を水平な床に立たせた状態の $\mathbf{R}_{WI}$ を 1 回記録して

$$\mathbf{R}_{BI} \leftarrow \mathbf{R}_{WI, \text{level}}^{\top} \text{ のロール・ピッチ成分} \quad (23)$$

とする零点合わせの方が確実である。立位で $\phi = \theta = 0$ になることを確認してから歩行に進む。ロール・ピッチは $\mathbf{R}_{WB}$ の成分から

$$\theta = \text{atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right), \quad \phi = \text{atan2}(r_{32}, r_{33}) \quad (24)$$

で取る。角速度と比力は $\boldsymbol{\omega}_B = \mathbf{R}_{BI}\boldsymbol{\omega}_I$ 、 $\mathbf{f}_B = \mathbf{R}_{BI}\mathbf{f}_I$ で胴体座標系に回す。角速度はジャイロ減衰の入力なので、フィルタを通さず生値を使い、必要なら 30 Hz の 1 次 LPF だけ掛ける。

メッセージの時刻 `header.stamp` と受信時刻の差を毎周期記録し、デバッグ配列に出す。これが理論ノート §4.3 で要求した経路遅れ T_d の実測になる。

6.2 重心と ξ

支持足の足裏中心を原点とし、実測関節角 $\mathbf{q}_m$ で FK を回して

$$\mathbf{x}_C = \mathbf{R}_{WB} (\mathbf{r}_C^B(\mathbf{q}_m) - \mathbf{r}_F^B(\mathbf{q}_m)) \quad (25)$$

の水平成分を取る。 $\mathbf{r}_C^B$ は最初は骨盤からの固定オフセットでよい。速度は

$$\dot{\mathbf{x}}_{C,k} = (1 - \beta) \dot{\mathbf{x}}_{C,k-1} + \beta \frac{\mathbf{x}_{C,k} - \mathbf{x}_{C,k-1}}{\Delta t}, \quad \beta = \frac{2\pi f_c \Delta t}{1 + 2\pi f_c \Delta t}, \quad f_c = 8 \text{ Hz} \quad (26)$$

で作り、 $\hat{\xi} = \mathbf{x}_C + \dot{\mathbf{x}}_C/\omega$ とする。歩の境界で支持足が変わると原点が跳ぶので、式 (3) で前周期の値を新しい座標系に移してから差分を取る。

sync read が周期内に収まらない場合は、指令角 $\mathbf{q}_{cmd}$ で FK を回す。このとき $\mathbf{r}_C^B - \mathbf{r}_F^B$ は計画値そのものだが、 $\mathbf{R}_{WB}$ が実測なので、胴体が傾けば $\mathbf{x}_C$ は計画からずれる。IMU だけでも「倒れかけている」ことは $\hat{\xi}$ に現れる。関節のたわみと足裏の滑りは拾えないので、読めるようになったら実測に戻す。

6.3 接地判定と位相リセット

スコアを

$$s_{td} = w_{load} [\text{遊脚側 膝・足首の負荷} > \text{閾値}] + w_{jerk} [|f_{B,z} \text{の高域成分}| > \text{閾値}] + w_{\phi} \text{ramp}(\phi; 0.6 \rightarrow 1.0) \quad (27)$$

とし、 $s_{td} \geq 1$ が 2 周期続いたら接地とする。重みは負荷 0.6、加速度 0.5、位相 0.5 から始める。位相 $\phi < 0.6$ での接地はつまずきと判定し、歩を中断して両脚支持の立位に戻す。 $\phi = 1.0$ で未接地なら降下を続け、 $\phi = 1.2$ で接地扱いにして境界処理へ進む。判定に使う負荷は sync read の周期に合わせて間引いて読む。

7 出力段

7.1 逆運動学への受け渡し

支持脚の足先目標は、支持足が床に固定されているので $\{L\}$ で

$$\mathbf{T}_{LF}^{\text{sup}} = \begin{pmatrix} \mathbf{R}_z(-\psi_P) & \mathbf{p}_i - \mathbf{r}_P \\ \mathbf{0}^\top & 1 \end{pmatrix} \quad (28)$$

である。 ψ_P は骨盤のヨーで、支持足座標系から見た骨盤の向きである。遊脚は軌道値をそのまま入れる。両脚とも $\mathbf{T}_{BF} = \mathbf{T}_{BL}\mathbf{T}_{LF}$ に直してから IK に渡す。IK は片脚ずつ独立で、股ヨー・ロール・ピッチ、膝、足首ピッチ・ロールの 6 角を返す。IK が到達域外を返したら、着地点を名目側へ縮めて $(\mathbf{p}_{i+1} \leftarrow \mathbf{p}_{i+1}^{\text{nom}} + 0.5(\mathbf{p}_{i+1} - \mathbf{p}_{i+1}^{\text{nom}}))$ その周期内で再計算する。

7.2 サーボ角と較正

関節角 $\mathbf{q}$ からサーボへは二段の変換を通す。

1. リンク変換：膝リンクと足首パラレルリンクの機構で、関節角からサーボ出力軸角へ。閉形式か LUT (IK タスクの成果物)
2. 較正：軸ごとに $\text{tick} = o_j + d_j \theta_j \frac{N}{2\pi}$ 。 o_j は零点オフセット、 $d_j = \pm 1$ は回転方向、 N は 1 回転のティック数 (型番で確認)

o_j, d_j は `calib/servo.yaml` に置き、立位の姿勢で各軸を 1 つずつ合わせて決める。関節リミットも同じファイルに置き、IK の出力をこれで検査する。

7.3 sync write と sync read

書くのは 18 軸の目標位置で、2 バスに分けて連続で送る。読むのは脚 12 軸の位置で、負荷は 4 周期に 1 回、電圧・温度は 1 秒に 1 回に間引く。バスの 1 往復に何 ms かかるかは既に用意した `feetech_benchmark.py` で測り、書き+読みが 5 ms に入らなければ、読みを隔周期にするか周期を 100 Hz に落とす。100 Hz でも振子の時定数の 13 倍で、歩行は成立する。

7.4 publish

`/joint_states` は出力バッファから 10 Hz のタイマで出す。名前は `l_hip_yaw` のように URDF と一致させる。`/motion/state` は状態・支持脚・位相を文字列で、状態が変わった周期にだけ出す。`/walk/debug` は配列で、順番を `walk_core` の定数で固定し、PlotJuggler の設定ファイルをリポジトリに置く。出す量は $\hat{\xi}$ 、計画 ξ 、 $\mathbf{p}_i$ 、 $\mathbf{p}_{i+1}$ 、 ϕ, θ 、 $\boldsymbol{\omega}_B$ 、足首オフセット、接地スコア、IMU 遅れ、周期遅れである。

8 リアルタイム性の確保

- RT スレッドは単調時計 (CLOCK_MONOTONIC) で絶対時刻のデッドラインを持ち、そこまで眠る。相対スリープは遅れが蓄積する
- プロセスを SCHED_FIFO の優先度で起動する (chrt -f 80)。Ubuntu では limits.conf (/etc/security/ 以下) で rtprio を許可しておく
- ROS の executor は別スレッドで回し、コールバックはメッセージの複製だけにする。Python では GIL があるので、コールバックの実行時間がそのまま RT スレッドの遅れになる。200 Hz の /imu/data コールバックで四元数の変換をしない
- 周期遅れ late の分布を最初の 1 週間は常に記録する。判定基準は「5 ms 周期で遅れの 99 パーセンタイルが 1 ms 未満」とし、超えるなら順に、コールバックの軽量化、isolcpus でコアを 1 つ空ける、PREEMPT_RT カーネル (RTOS 導入タスク)、C++ 移植、の順で対処する
- bag 記録は別プロセスで行い、motion ノードからは /walk/debug を BestEffort で出すだけにする

9 調整とデバッグの導線

ゲインは実行時にパラメータで変える。

```
ros2 param set /motion gait.k_d_ankle "[0.05, 0.05]"
```

パラメータのコールバックは値の範囲を検査してスナップショットに書き、RT スレッドが次周期で拾う。層ごとの有効化は次の順で、各段に「見るもの」を決めておく。

段	有効化するもの	手順	見るもの
0	なし	立位、押して戻ることを確認	ϕ, θ の減衰波形
1	k_d_ankle	振動が出る手前まで上げ、2~3 割戻す	押し試験の整定時間
2	k_p_ankle	傾斜板に載せて定常傾きが減ること	θ の定常値
3	k_torso	0 から上げる	カメラ画像の水平、 ϕ, θ
4	足踏み ($v = 0$ で STEP 強制)	フィードフォワードのみ	$\hat{\xi}$ と計画 ξ の差
5	静歩行相当の低速前進	T を長く、 L_x を小さく	着地時刻の予定からのずれ
6	接地判定	閾値校正、位相リセット有効化	接地スコアの波形
7	k_dcm (横のみ)	ξ の出どころを推定値に	横押し試験の回復歩数
8	k_dcm (前後)	同上	前後押し試験
9	torque_soft	ON/OFF 比較	着地時の加速度スパイク

各段で /walk/debug を bag に残し、前段との比較を数値で持つ。

着地の前後は減衰項が衝撃を拾うので、予定着地の `gate_pre_td` 前から接地確認 + 20 ms までゲインを半分にする。このゲートは位相と接地判定から毎周期決め、デバッグ配列に出す。

ROS なしの単体テストは `walk_core` に対して次を回す。

- 生成した $\mathbf{p}(t)$ と $\mathbf{x}_C(t)$ が $\ddot{\mathbf{x}}_C = \omega^2(\mathbf{x}_C - \mathbf{p})$ を満たすこと（数値微分で残差を見る）
- 各周期で $\mathbf{p}(t)$ が支持足の足裏内にあること
- START $\rightarrow$ STEP $\rightarrow$ STOP の一連で ξ が発散せず中点に収束すること
- 同じ入力列で同じ出力列が出ること（決定性）
- IK の往復 $\text{FK}(\text{IK}(\mathbf{T})) = \mathbf{T}$ が可動域グリッド全点で通ること（IK タスクの試験をそのまま流用）

10 安全

- `/estop` は受信した周期のうちにトルクオフ。復帰は手動
- `/cmd_walk` の途絶 (`cmd_timeout`) は $v = 0$ とみなし STOP へ
- 転倒判定 (`fall_tilt`) で FALL へ。保護姿勢の技はモーション再生と同じ経路で出す
- サーボの応答途絶が 3 周期続いたら歩行を止めて立位へ。エラーは `/motion/state` に出す
- 起動直後はトルクオフで始め、`home` 技で立位に入ってから STEP を許可する
- 歩行を開始する前に非常停止の経路を必ず先に作る（アーキテクチャ文書 §5 の順序）

11 実装の順番

1. `walk_core` の骨組み：入出力の型、状態機械、パラメータ読み込み。単体テストの雛形
 2. DCM・重心・遊脚の生成（式 (5)~(12)）と、ZMP 整合の単体テスト
 3. IK タスクの成果物を接続し、足先目標 $\rightarrow$ サーボ角の往復テスト
 4. `motion` ノード：スナップショット、RT スレッド、`sync write`、`/joint_states`。周期遅れの計測
 5. 立位・`home`・非常停止の経路
 6. IMU 経路： $\mathbf{R}_{BI}$ の零点合わせ、 ϕ, θ の確認、遅れの実測
 7. 足踏み（段 4）、低速前進（段 5）
 8. 推定器： $\hat{\xi}$ 、接地判定
 9. フィードバックを段 1 から順に有効化
- 1~3 はロボットなしで進む。4 以降は実機で、各段の「見るもの」を bag に残す。

12 未確定・要実測

項目	決まらないと困る箇所	手段
バス 1 往復の時間	周期 (200/100 Hz)、sync read の間引き	<code>feetech_benchmark.py</code>
IMU 経路の遅れ T_d	<code>k_d_ankle</code> の上限、imu_filter の置き場所	§6.1 の記録
R_{BI}	姿勢の零点	立位で記録
サーボの分解能 N 、レジスタ (トルクリミット、負荷)	較正、着地剛性	型番のレジスタ表
w_{hip} , l_{th} , l_{sh} , 足裏寸法	L_x 上限、クランプ、 W	CAD (IK タスク)
周期遅れの分布	Python 継続か C++ 移植か	§8 の基準
接地の負荷・加速度波形	閾値	実機で足踏みして採る